

Algorytmy metaheurystyczne, lista 3

Błażej Pawluk

08.062026

1 Opis algorytmu

1.1 Wyznaczanie populacji początkowej

Każdy osobnik na początku jest losową permutacją. Osobników jest `popSize = 100`.

1.2 Selekcja

Turniej: w turnieju z `k = 10` losowych osobników wybieramy 2 o najlepszym wyniku, którzy zostaną rodzicami 2 nowych osobników w następnej generacji. Turniej jest następnie wykonywany tyle razy, żeby w nowej generacji było tyle samo osobników co w obecnej. Dzięki turniejowi tylko najlepsze osobniki zostają rodzicami, czasami kilkukrotnie, a najgorsze nigdy nie zostają rodzicami.

1.3 Krzyżowanie

Rodzice P1 i P2. Rodzic P1 przepisuje do potomka fragment permutacji o losowej długości z przedziału `[n * crossSizeMin=0.4, n * crossSizeMax=0.7]` od losowego wierzchołka, następnie P2 wykonuje jedną z możliwości: Order crossover i Partially mapped crossover, po czym rodzice zamieniają się miejscami i wykonują to samo, aby powstał drugi nowy osobnik.

1.3.1 Order crossover

Po przepisaniu fragmentu od pierwszego rodzica, pozostałe geny są uzupełniane w kolejności występowania w drugim rodzicu z pominięciem duplikatów.

1.3.2 Partially mapped crossover

Po przepisaniu fragmentu od pierwszego rodzica, dla każdego genu z drugiego rodzica wyznaczana jest ścieżka mapowań prowadząca do wolnej pozycji, poza przepisany segmentem.

1.4 Mutacja

Z prawdopodobieństwem `mutationProbability = 0.3` zachodzi mutacja - `invert` od losowego wierzchołka o losowej długości z zakresu `[n * mutationSizeMin=0.1, n * mutationSizeMax=0.2]`.

1.5 Warunek stopu

Jako warunek stopu przyjmujemy stałą liczbę generacji, która zostanie przesymulowana, w moim algorytmie `generations = 200`.

1.6 Algorytm wyspowy

Populacja podzielona jest na `islands = 4` wysp. Co pewną liczbę generacji (`generations / migrationIntervalRate=10 = 20`) następuje migracja między wyspami: 3 osobników o najlepszym wyniku oraz 2 innych losowych przechodzi na $(i+1) \% islands$ wyspę.

1.7 Współbieżność

Zarówno dla wyznaczania populacji początkowej, jak i późniejszej symulacji, każda wyspa wykonuje operacje równoległe na własnym wątku.

1.8 Algorytm memetyczny

Zarówno przy wyznaczaniu populacji początkowej, jak i po krzyżowaniu i ewentualnej mutacji każdy osobnik przechodzi przez `adolescenceLength = 5` kroków algorytmu `LocalSearch`.

2 Wyniki algorytmu genetycznego

2.1 Średnie ścieżki znalezione przez algorytm

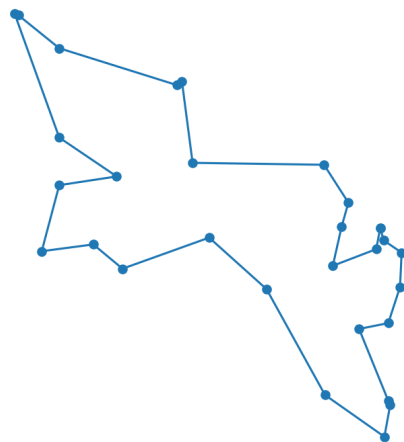
dataset	GC-OC	GC-PMC
wi29	27603	27603
dj38	6656	6656
qa194	9544.1	9557
uy734	145942	145806
zi929	194125	194574
mu1979	1133030	1131110

Tabela 1: Porównanie ścieżek znalezionych przez algorytm z różnymi metodami krzyżowania: OC - Order Crossover, PMC - Partially Mapped Crossover

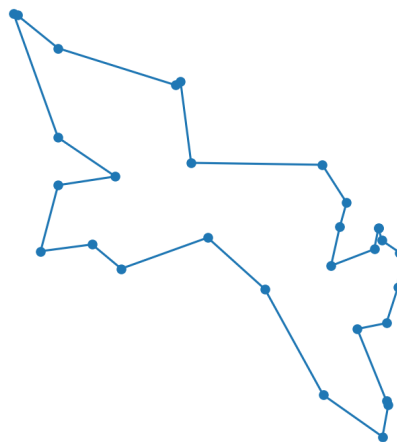
Wyniki obu sposobów krzyżowania są bardzo zbliżone. W przypadku 2 najmniejszych zbiorówoba algorytmy za każdym razem znalazły optymalne rozwiązania

2.2 Najlepsze ścieżki znalezione przez algorytm

2.2.1 wi29

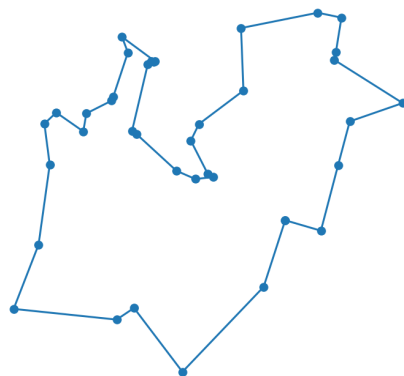


(a)

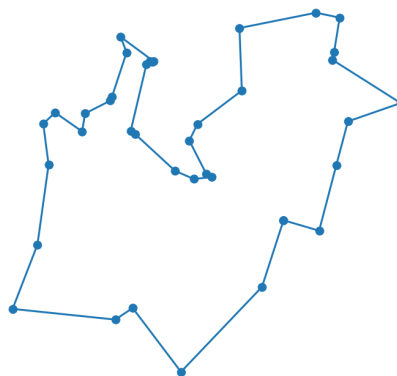


(b)

2.2.2 dj38

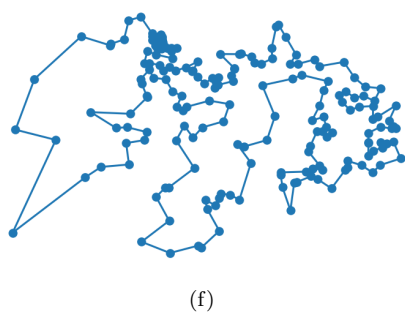
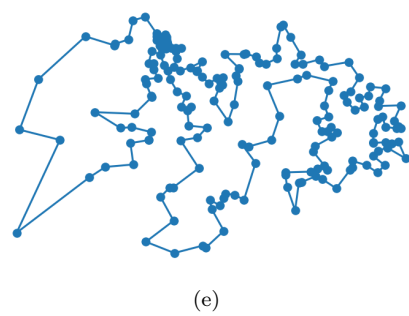


(c)

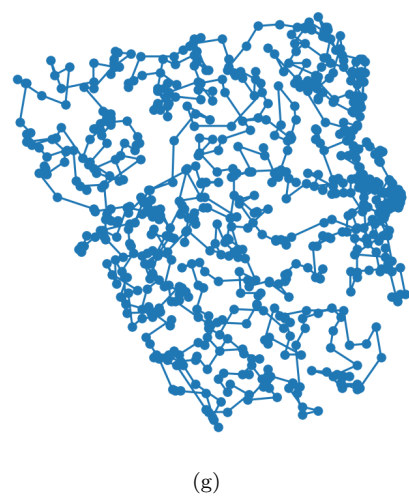


(d)

2.2.3 qa194



2.2.4 uy734



2.2.5 zi929

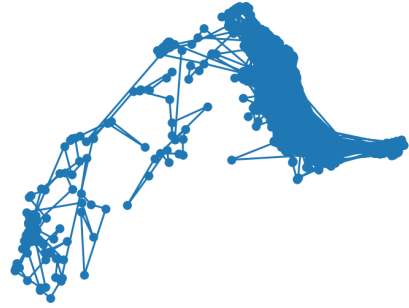


(i)

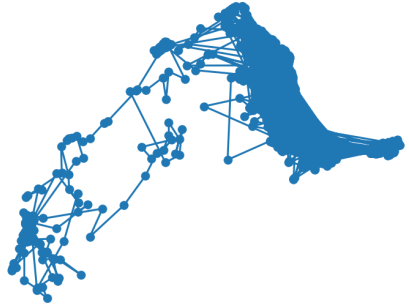


(j)

2.2.6 mu1979



(k)



(l)

3 Porównanie wszystkich algorytmów

dataset	LS	RLS	LSMST	SA	TS	GS-OC	GS-PMC
wi29	28271	42582.3	28173.2	27901.4	27625.5	27603	27603
dj38	6904	11605.3	6926.86	6785.53	6742	6656	6656
qa194	10152.9	16947.2	9912.43	10209.6	10048	9544.1	9557
uy734	86931.1	174026	84422.1	89475.3	88823.2	145942	145806
zi929	104192	155496	101955	107683	104527	194125	194574
mu1979	94191.7	182995	93405.7	98133.2	96405.7	1133030	1131110

Tabela 2: Porównanie średnich ścieżek znalezionych przez algorytmy rozwiązywania TSP (LS - LocalSearch, RLS - RandomLocalSearch, LSMST - LocalSearch z wykorzystaniem MST, SA - Simulated Annealing, TS - TabuSearch, GS-OC - GeneticSearch z OC, GS-PMC - GeneticSearch z PMC)

Dla mniejszych danych najlepsze wyniki otrzymywał algorytm genetyczny, jednak dla większych najlepszy był Local Search wykorzystujący minimalne drzewo rozpinające. LSMST był też najszybszy. Aby GeneticSearch działał dobrze też dla większych zbiorów należałoby dodać więcej generacji - liczbę rzędu liczby wierzchołków razy, jednak wtedy długość działania algorytmu jeszcze bardziej by się wydłużyła.